Obligatorio 2

Grupo 21 - Tutor: Matias Valdés
Obligatorio 2 - Métodos Numéricos 2022

Matías Repetto 5160290-4
Facundo Silva 4518448-5
Javier Iglesias 5139337-9
Germán del Castillo 4982824-3
Andrés de la Puente 5203134-0
Grupo 21 - Tutor: Matias Valdés
Obligatorio 2 - Métodos Numéricos 2022.

18 de Noviembre de 2022

Índice

1. Introducción **2**

2. Marco Teórico **2**

3. Desarrollo **5**
3.1. Resolucion Analitica PVI 5
3.2. Resolución mediante Heun 8
3.2.1. Paso a paso del Algoritmo de *Heun* 8
3.3. Resolución mediante comando del lenguaje utilizado (LSODE) 10
3.4. Comparación de solución entre métodos de Heun, Lsode y Solución Exacta. . 11
3.5. Interpolación con splines cúbicos 16
3.6. Mínimos Cuadrados No Lineal 18

4. Conclusiones **20**

Referencias **21**

1. Introducción

El objetivo de este obligatorio es acercarnos a la solución de una ecuación diferencial de la forma:

$$y'(t) = f(t, y) \tag{1}$$

Con condición inincial $y(t_0) = y_0$

Esto se logrará mediante la implementación de distintos métodos numéricos que serán explicados a lo largo de este documento.

La ecuación a resolver es la siguiente:

$$y'(t) = y^2(t)(1 - 3t) - 3y(t) \tag{2}$$

Se obtendrán soluciones para las diferentes condiciones iniciales (4 distintas en este caso) y se observará su distinto comportamiento debido a estas.

Los métodos para aproximarnos nos devolverán los valores $y_n(t_n)$ en dónde (t_n) son diferentes tiempos t conocidos. La idea es que estos valores de $y_n(t_n)$ se acerquen a la solución real en (t_n) es decir $y(t_n)$.

Luego también con ótros métodos se obtendrán funciones que aproximen los resultados obtenidos previamente. Finalmente, dichas soluciones se podrán comparar al resultado que se obtine utilizando una función provista por Octave, que permite resolver este tipo de PVI (Problema de Valor Inicial).

2. Marco Teórico

- **Método de Heun**

  El método se basa en utilizar la pendiente en cada punto, de la solución a la ecuación diferencial. Por la forma de la ecuación, sabemos que $f(t, y)$ es la derivada de y(t) en el punto (t,y) y por ende la pendiente de la recta tangente en ese punto. Si se tienen un conjunto de valores de t equidistantes (distancia = h) con el método de Euler se aproxima el valor de $y(t_{n+1})$ con un incremento h en t y un incremento en y de $h * f(t_n, y_n)$ siendo f la pendendinte dy/dt en (t_n, y_n) (Euler hacia adelante). El caso de Euler hacia atras es análogo solo que utilizando como pendiente a $f(t_{n+1}, y_{n+1})$.

  Como se sabe que $y(t_0) = y_0$ y utilizando la ecuación de una recta dada su pendiente y un punto de ella, la iteración queda de la siguiente manera:

  Para Euler hacia adelante

  $$y(t_0) = y_0 \tag{3}$$

  $$y_{n+1} = y_n + f(t_n, y_n) * h \tag{4}$$

  Para Euler hacia atrás

  $$y(t_0) = y_0 \tag{5}$$

  $$y_{n+1} = y_n + f(t_{n+1}, y_{n+1}) * h \tag{6}$$

El método de Heun mejora al de Euler ya que toma, en vez de la pendiente $f(t_n, y_n)$ o $f(t_{n+1}, y_{n+1})$, un promedio de ambas. Es decir una pendiente $(f(t_{n+1}, y_{n+1}) + f(t_n, y_n))/2$. Quedando la nueva iteración entonces:

$$y(t_0) = y_0 \tag{7}$$

$$y_{n+1} = y_n + (h/2) * (f(t_{n+1}, y_{n+1}) + f(t_n, y_n)) \tag{8}$$

De esta ecuación queda despejado $, y_{n+1}$ siempre que f es lineal en $, y_{n+1}$. En caso contrario se aplica Euler hacia adelante en el segundo miembro de la ecuación, dentro la función f quedando:

$$y_{n+1} = y_n + (h/2) * (f(t_{n+1}, y_n + f(t_n, y_n) * h) + f(t_n, y_n)) \tag{9}$$

- **Interpolación mediante splines cúbicos**

  Los métodos de interpolación son métodos númericos para aproximar una función a partir de una serie de puntos. Estos métodos cobran sentido cuando se obtiene una muestra experimental y se quiere encontrar una función que describe lo observado. Una de las variantes de estos es la interpolación por splines cúbicos que es un método de interpolación a trozos del tipo iterativo.

  La interpolación con splines a diferencia de otras variantes tiene como objetivo no solo aproximar los valores de la función para que sea continua, sino que también busca que las derivadas primera (pendiente) y segunda (concavidad) existan y sean continuas.

  Una vez se tiene un conjunto de puntos (t,y) que safistacen una cierta función y(t), la idea es obtener funciones $s_n(t)$ polinómicas de 3er grado en cada intervalo entre 2 puntos consecutivos (t_n, y_n) y (t_{n+1}, y_{n+1}). Se pedira por supuesto que las funciones s_n pasen por (t_n, y_n) y (t_{n+1}, y_{n+1}). Además se quiere que $s'_n(t_n) = s'_{n+1}(t_n)$ y que $s''_n(t_n) = s''_{n+1}(t_n)$.

  En forma de sistema de ecuaciones, estas son el conjunto de condiciones que tienen que cumplir los splines:

$$\begin{cases} a_{i0} + a_{i1}x_i + a_{i2}{x_i}^2 + a_{i3}{x_i}^3 = y_i(x_i) \\ a_{i0} + a_{i1}x_{i+1} + a_{i2}{x_{i+1}}^2 + a_{i3}{x_{i+1}}^3 = y_{i+1}(x_{i+1}) \\ a_{i1} + 2a_{i2}x_i + 3a_{i3}{x_i}^2 = s'_i(x_i) \\ a_{i1} + 2a_{i2}x_{i+1} + 3a_{i3}{x_{i+1}}^2 = y_{i+1}(x_{i+1}) \end{cases}$$

- **Método de ajuste por mínimos cuadrados no lineales**

  Se tienen una serie de datos t_n, y_n. El objetivo es encontrar la función $f(t, x_1, x_2..., x_n)$, siendo $x_1, x_2..., x_n$ los parametros de la función a encontrar, que mejor ajusta a los puntos (t_n, y_n) siguiendo cierto criterio de distancia que se verá a continuación.

  Le llamamos X al vector de parámetros $X = (x_1, x_2..., x_n)$ y $F(X)$ a la siguiente matriz:

$$\begin{pmatrix} f(X,t_1) \\ f(X,t_2) \\ \vdots \\ f(X,t_n) \end{pmatrix}$$

Luego Y es:

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Se define el residuo R como $R(X) = F(X) - Y$ y la idea es minimizar R tomando como tamaño de vector a la norma 2. Esto se reduce a encontrar $min_{\vec{\mathbf{x}} \in \mathbb{R}^n} = ||F(\vec{\mathbf{x}}) - Y||_2^2$.

Como queremos resolver el problema cuando F no es lineal en X, se utiliza el algoritmo de Gauss Newton que permite convertir el problema a uno lineal. Una vez llevado a forma lineal, se supondrá que se tiene una buena aproximación de la solución (X_k) para utilizar el siguiente método iterativo que consiste en econtrar:

$$min_{P_k \in \mathbb{R}^n} = ||R(X_k) + J_R(X_k)P_k||_2^2$$

Siendo $P_k = X_{k+1} - X_k$, es decir que si queremos X_{k+1} es simplemente $X_{k+1} = X_k + P_k$ Más adelante en la explicación del algoritmo usamos $J_F(X_k)$, siendo este $J_R(X_k) = -J_F(X_k)$.

3. Desarrollo

3.1. Resolucion Analitica PVI

Lo primero será analizar la ecuacion diferencial y reacomodar sus componentes buscando ciertos patrones que nos ayuden a encarar la resolucion de la misma, entonces como problema principal se presenta el siguiente problema de valores inciales por su siglas PVI.

Sea un PVI en $[-1, 2]$ y la siguiente ecuacion diferencial :

$$y'(t) = y^2(t)(1 - 3t) - 3y(t) \tag{10}$$

$$y(-1) = y0 \tag{11}$$

Donde la condicion incial del problema se obtuvo, repetidas veces, mediantes mediciones resultando en la siguiente tabla:

Mediciones de y0
0.9142
0.8655
1.0861
1.0832

Se debe mencionar que tambien se cuenta con un conjunto de puntos de tiempos proporcionados para la resolucion de esta tarea, dados en los archivos *tiempos.mat* y *tiempos*2*.mat*.

El siguiente paso fue analizar propiamente la ecuacion diferencial, en este punto nos dimos cuenta que dicha ecuacion tiene la forma de una ecuacion diferencial de *Bernoulli* debido a que tras un pequeño re acomodo podemos lograr la forma que coracteriza a dicho tipo de ecuaciones.

Para dar un poco mas de contexto las ecuaciones diferenciales ordinarias se dicen de *Bernoulli* si las mismas tienen una forma tal que :

$$y' + P(x)y = Q(x)y^n \tag{12}$$

Esta forma en particular nos facilita la resolucion debido a que justamente *Bernoulli* dice que si $n \neq 0$ y $n \neq 1$ entonces tenemos un cambio de variable tal que $u = y^{1-n}$ con dicho cambio de variable podemos llegar a transofrmar nuestra ecuacion a una ecuacion diferencial lineal. En particualr en nuestro caso logramos re ordenar para que la misma quedara como:

$$y'(t) + 3y(t) = (1 - 3t)y^2(t) \tag{13}$$

lo cual acorde a la forma de *Bernoulli* tendriamos a $P(x) = 3$ y $Q(x) = (1 - 3t)$, una vez identificado esto el cambio de variable que asumiriamos sera $u = y^{1-n} = y^{1-2} = y^{-1}$, entonces desarollamos esto mismo.

Con la forma *Bernoulli* aplicada procedemos a re escribir la ecuacion :

$$\frac{dy}{dt} + 3y = (1-3t)y^2 \tag{14}$$

Luego

$$u = y^{-1} \tag{15}$$

entonces

$$\frac{dy}{dt} = \frac{d(u^{-1})}{du}\frac{du}{dt} \tag{16}$$

por ende

$$\frac{dy}{dt} = -u^{-2}\frac{du}{dt} = \frac{-1}{u^2} \tag{17}$$

Sustituimos a y por u y a $\frac{dy}{dt}$ por $\frac{du}{dt}$ y queda de la siguiente forma

$$-u^{-2}\frac{du}{dt} + 3u^{-1} = (1-3t)u^{-2} \tag{18}$$

$\Longrightarrow$

$$\frac{-1}{u^2}\frac{du}{dt} + \frac{3}{u} = \frac{(1-3t)}{u^2} \tag{19}$$

$\Longrightarrow$

$$(\frac{-1}{u^2}\frac{du}{dt} + \frac{3}{u})u^2 = (1-3t) \tag{20}$$

$\Longrightarrow$

$$-\frac{du}{dt} + 3u = (1-3t) \tag{21}$$

acomodamos signos a conveniencia por estetica, $\Longrightarrow$

$$u' - 3u = -(1-3t) \tag{22}$$

en este punto identificamos que para resolver esta ecuacion diferencial lineal que nos queda podemos aplicar el metodo de $FactorIntegrante$, no es menor aclarar que para poder aplicar este metodo necesitamos que la ecuacion a resolver sea de la forma $\frac{dy}{dx} + P(x)y = f(x)$ siendo que $P(x)$ puede ser una constante que multiplica a y y que $f(x)$ puede ser una constante, luego la formula de este factor integrador sera $FI = e^{\int P(x)dx}$.

Aplicando este metodo entonces tomamos $P(x) = -3$ y $f(x) = -(1-3t)$ por lo tanto en nuestro caso el factor integrante sera:

$$FI = e^{\int -3dt} \tag{23}$$

$\Longrightarrow$

$$FI = e^{-3t} \tag{24}$$

$\Longrightarrow$ multiplicamos FI por la ecuacion diferencial

$$e^{-3t}\frac{du}{dt} - e^{-3t}3u = -e^{-3t}(1-3t) \tag{25}$$

$\Longrightarrow$

$$\frac{d(e^{-3t}u)}{dt} = -e^{-3t}(1-3t) \tag{26}$$

$\Longrightarrow$

$$\int \frac{d(e^{-3t}u)}{dt}dt = \int -e^{-3t}(1-3t)dt \tag{27}$$

$\Longrightarrow$

$$e^{-3t}u = -e^{-3t}t + C \tag{28}$$

$\Longrightarrow$

$$u = -t + Ce^{3t} \tag{29}$$

$\Longrightarrow$

$$u = y^{-1} \tag{30}$$

$\Longrightarrow$

$$y^{-1} = -t + Ce^{3t} \tag{31}$$

$\Longrightarrow$

$$y(t) = \frac{-1}{Ce^{3t} + t} \tag{32}$$

$\Longrightarrow$ tomando en cuenta la condicion inicial y que tenemos 4 valores de $y0$ esto nos dejara con una tabla como la siguiente :

$y0$	C
0,9142	$\frac{-e^3}{0,9142} + e^3$
0,8655	$\frac{-e^3}{0,8655} + e^3$
1,0861	$\frac{-e^3}{1,0861} + e^3$
1,0832	$\frac{-e^3}{1,0832} + e^3$

Se observa que para $y_0 < 1$ tenemos $C < 0$ y para $y_0 > 1$ tenemos $C > 0$. Analizando la función $Ce^{3t} + t$ en Geogebra se observa que tiene raiz si y solo si $C >= 0$. Por lo tanto, si $y_0 > 1$ ($C > 0$) tenemos que la solución $y(t)$ tiene un valor $t = t_a$ tal que $Ce^{3t} + t = 0$, por lo que t_a no pertenece al dominio de la función. En otras palabras, se tiene una asíntota vertical en un cierto $t = t_a$.

Por el contrario, si $y_0 > 1$ la solución $y(t)$ tiene de dominio a todos los Reales.

3.2. Resolución mediante Heun

En este punto empezamos a desarrollar la solución de Heun para octave, como ya introdujimos en este documento esta es muy parecida a la solucion de *Euler* pero este mismo mejora la implementación del ya antes mencionado $Heun$, es esencialmente un método $Rugne-Kutta$ de segundo orden también conocido como $EulerMejorado$, vale nombrar que $Heun$ es un metodo implícito, por lo tanto, salvo que f sea lineal en x, necesitamos resolver una ecuacion no lineal.

Para poder lidear con esto generalmente lo que se hace es tomar un predictor, esto es imponer un paso previo x^*_{k+1} para luego determinar x_{k+1}. Tenemos asi que $Heun$ es un ejemplo de método multi-paso predictor-corrector, dejando al cuerpo iterador del metodo de la siguiente manera :

$$\begin{cases} Prediccion: \ x^*_{k+1} = x_k + hf(t_k, x_k), \\ Correcion: \ x_{k+1} = x_k + \frac{h}{2}[f(t_k, x_k) + f(t_{k+1}, x^*_{k+1})] \end{cases} \tag{33}$$

Aqui se calcula la prediccion usando el metodo de $Euler$.

3.2.1. Paso a paso del Algoritmo de $Heun$

Antes de especificar el método es necesario recalcar que en este punto se trabaja con el archivo *tiempos.mat* que se da como datos para realizar la tarea. A dicho archivo se le agregan los valores $[-1, 2]$ para posterior estudio más adelante, entonces como primer paso definimos la parte derecha de la Ecuacion Diferencial Ordinaria (EDO):

$$f = y^2(t)(1 - 3t) - 3y(t) \tag{34}$$

Como segundo paso debemos en este momento definir pasos iniciales y finales de evaluacion, en particular en nuestro caso no es necesario ya que como antes describimos recibimos un archivo con un arreglo de valores de tiempo, entonces lo unico que tendriamos que hacer es ordenarlo ascendentemente para luego porder formar el salto al momento de iterar el método.

Lo único que resta es iterar la *Predicción* y *Corrección* antes mencionada, con el correspondiente paso h calculado como el valor absoluto de la diferencia de $tiempos(i+1) - tiempos(i)$ dando como resultado el siguiente loop.

```
For i=1:n-1
    h = abs(tiempos(i+1)-tiempos(i))
    p(i+1) = y(i) + h*f(tiempos(i), y(i))
    y(i+1) = y(i) + (h/2)*(f(tiempos(i), y(i))+f(tiempos(i+1),p(i+1)))
endfor
```

Comentar que llegado al punto de experimentacion cpn este algoritmo nos dimos cuenta que para ciertos valores iniciales este problema presentaba singularidades que hacian que la solucion se dispare a valores infinitos, para dar mas contexto en particular en problemas de valores iniciales PVI podemos llegar a tener puntos de interes que resultan en puntos singulares llamados *Puntosdeimpasse* y *Puntosdecruce*.

Un punto de impasse es un punto singular en el cual una solucion colapsa en tiempo finito con velocidad infinita, a esta curva se la llama solucion de impasse. Mas especificamente, si la curva "llega" al punto se llama punto de impasse de entrada y, si "sale" del punto, punto de impasse de salida. Un punto de cruce es un punto singular por el cual pasa al menos una curva solucion, a la curva se la llama solucion de cruce.

En nuestro caso particular presentabamos singularidades con las condiciones iniciales $y0 = 1{,}0861$ y con $y0 = 1{,}0832$ por ende lo que procedimos a hacer es en un principio operar la iteracion de $Heun$ con la ecuacion diferencial pero con el cambio de variable $u = y^{-1}$. Esto quedaria que en la parte de iteracion de $Heun$ utilizariamos la funcion :

$$faux = -(1 - 3t) + 3y \tag{35}$$

Esto nos dara un vector resultado al cual debemos devolver el cambio de variable que hicimos en un principio, entonces sea yCv el vector resultado de el metodo de $Heun$ con el cambio de variable entonces la solucion final sera $yS = \frac{1}{yCv}$. Visualizacion de la iteracion.

```
For i=1:n-1
    h = abs(tiempos(i+1)-tiempos(i))
    p(i+1) = y(i) + h*faux(tiempos(i), y(i))
    y(i+1) = y(i) + (h/2)*(faux(tiempos(i), y(i))+faux(tiempos(i+1),p(i+1)))
endfor
y = 1/.y
```

Por lo cual con estos arreglos logramos bypassear las singularidades, mas adelante en este documento se veran de forma grafica estas soluciones.

3.3. Resolución mediante comando del lenguaje utilizado (LSODE)

El comando utilizado por octave para la resolución del PVI es lsode (Livermore Solver for Ordinary Equations) el cual forma parte del paquete ODEPACK de Alan C. Hindmarsh. Este puede resolver ecuaciones diferenciales ordinarias de la forma $\frac{dx}{dt} = f(x,t)$ con $x(t_0) = x_0$.

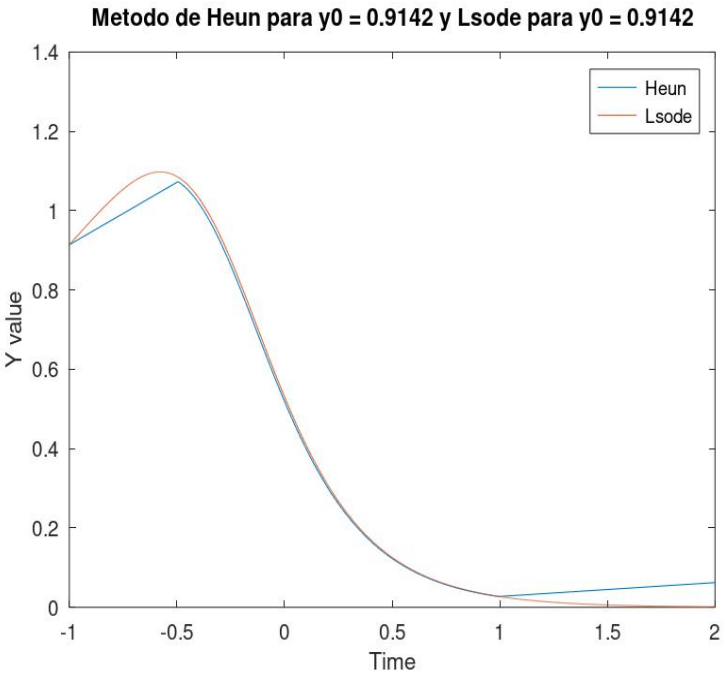
A la hora de utilizar la funcion, se llama de esta manera: $[x, istate, msg] = lsode(f, x_0, t)$, en donde 'x' es un array con las soluciones en los tiempos 't', 'istate' es una bandera que indica si salta si la solucion se computo bien, y 'msg' da mas informacion en caso de que istate salte. Se le entregan 3 parametros, 'f' que es la funcion a resolver escrita en string, x_0 que es la solucion para el tiempo t_0, y "t"que es el array de tiempos para el cual se quiere hallar soluciones.

Dependiendo del sistema que se introduzca al comando se elige, de un grupo de posibles algoritmos, el método que mejor se ajuste al caso. En el caso de sistemas no rígidos utiliza métodos con formulas de corrector-predictor de Adams (método de $Heun$) y métodos de fórmulas de diferenciación hacia atrás (método de $Euler$ hacia atrás) en el caso de sistemas rígidos. En el caso de que el sistema sea lineal se utilizan métodos directos.

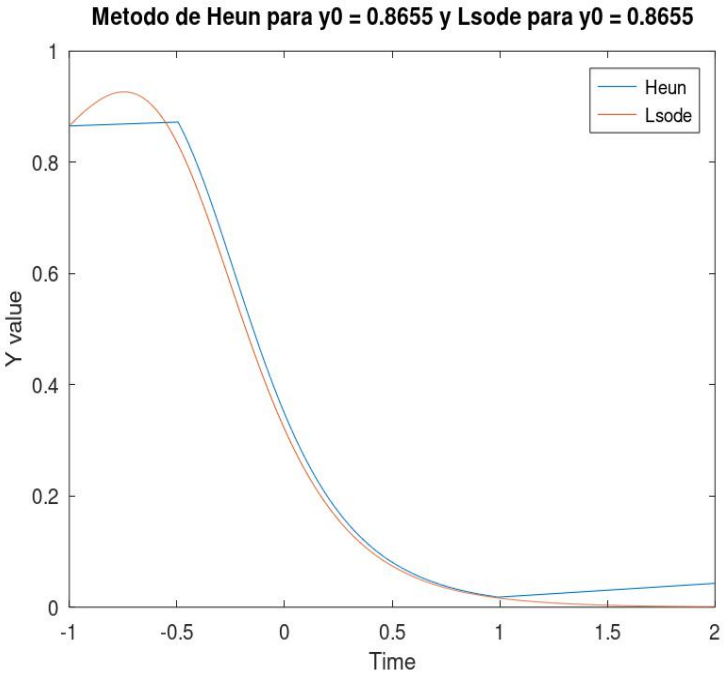
3.4. Comparación de solución entre métodos de Heun, Lsode y Solución Exacta.

En esta parte tenemos una comparacion grafica de los resultados arrojados por los distintos metodos y la solucion analitica al problema. En primera instancia estaremos comparando las graficas de $Heun$ y $Lsode$ particularmente para ver la diferencia de presicion entre los dos metodos :

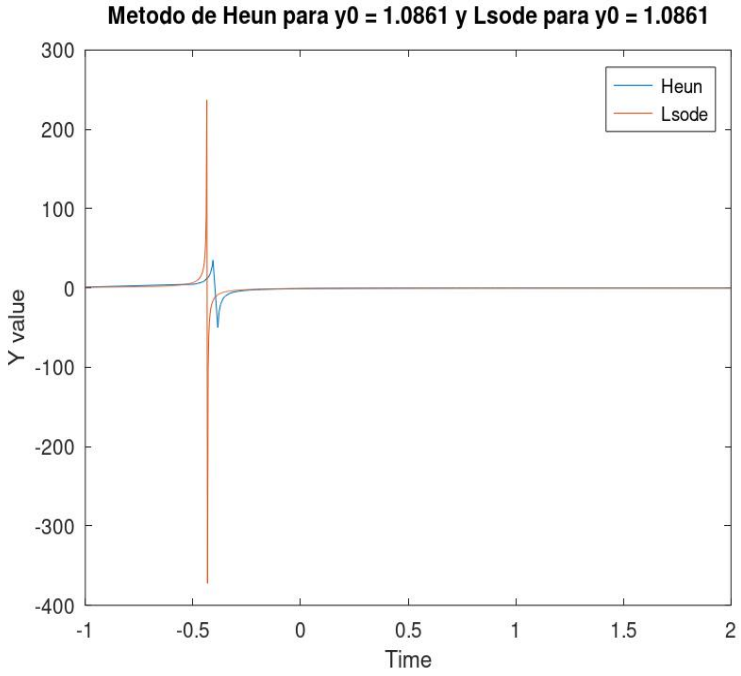
En primera instancia aqui tenemos la grafica solucion de Heun y Lsode con condicion inicial $y0 = 0{,}9142$ donde se puede ver que la solucion de *heun* ajusta considerablemente bien a la solucion presentada por el comando $Lsode$, tambien logramos ver que desde los tramos $[-1, -0{,}5]$ y $[1, 2]$ tenemos una suerte de diferencia muy grande, esto pasa porque en nuestro caso los datos de *tiempos.mat* no contaban con valores en estos intervalos, dicha irregularidad se puede solucionar agregando puntos entre estos intervalos. Contar que justamente $Lsode$ da un resultado mas armonioso entre dichos intervalos debido a que este comando opera con *linspace* este genera una cantidad de puntos uniformes entre los intervalos esto hace que por ende la ejecucion de $Lsode$ quede con mas puntos entre $[-1, -0{,}5]$ y $[1, 2]$.

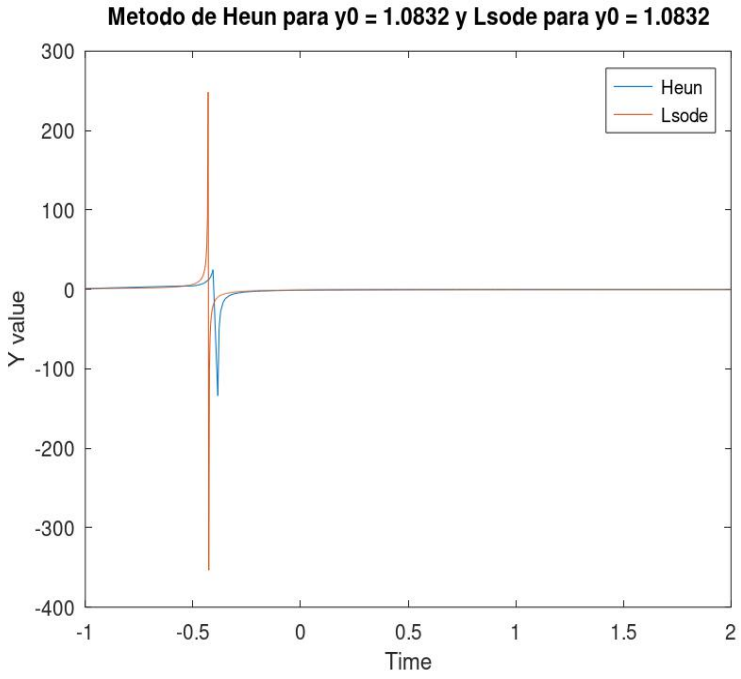


A continuacion podemos ver la grafica de solucion de Heun y Lsode con condicion inicial $y0 = 0{,}8655$, este caso se comporta muy similar a el primero $y0 = 0{,}9142$ pero en particular aqui podemos ver que ajusta un poco peor en general, reiterar que *Lsode* usa como paso una coleccion generada por un *linspace* esto hace que la solucion como tal por medio de este comando logre mas definicion que el *Heun* iterado en la coleccion de tiempos proporcionada.

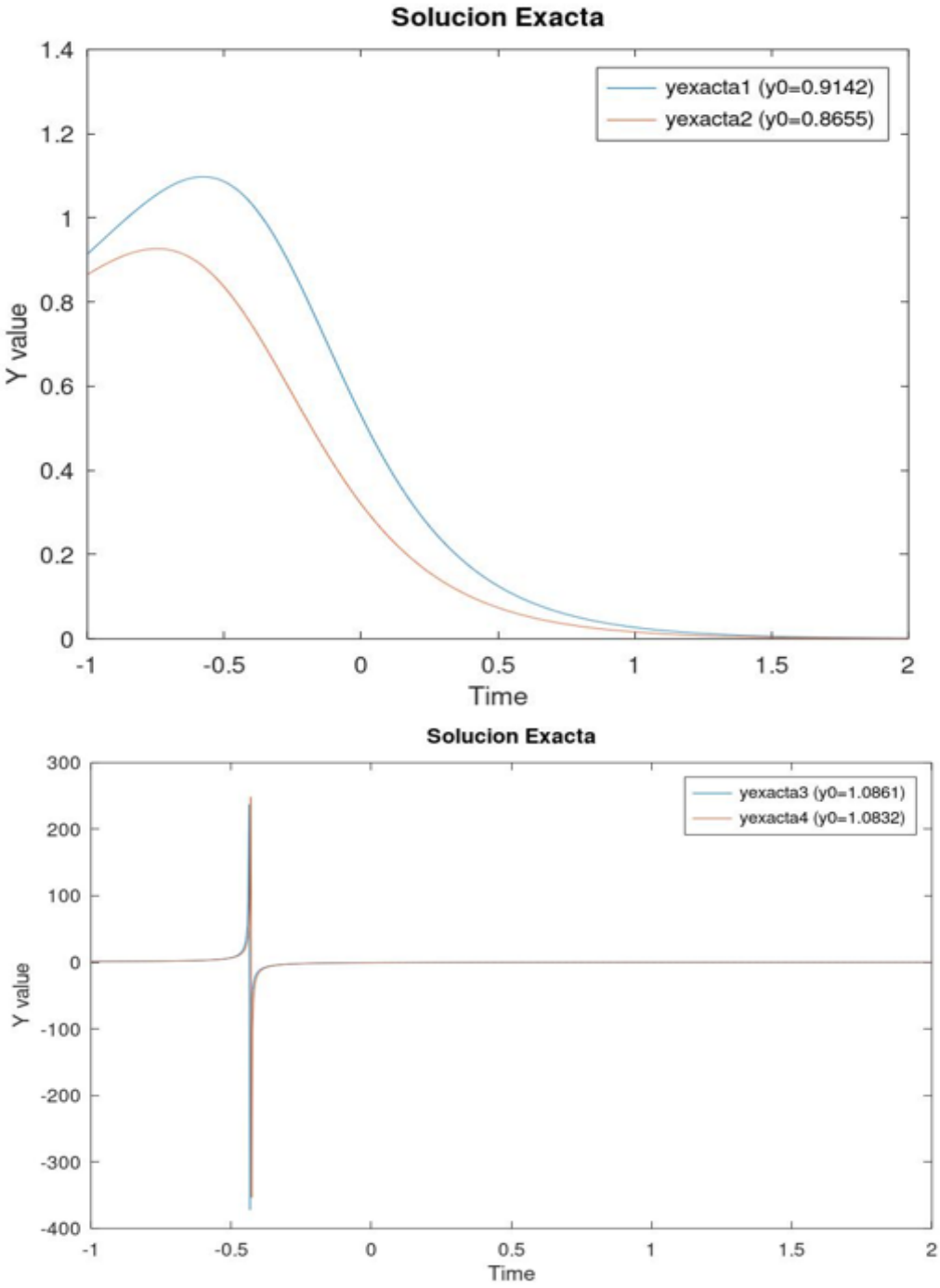


A continuacion presentaremos las graficas de solucion de Heun y Lsode para ambas condiciones iniciales $y0 = 1{,}0861$ y $y0 = 1{,}0832$ en particular en dichas graficas lo que mas llama la atencion es la especie de .ªsintota"vertical que forma esto se debe a una singularidad de impasse ya antes descrita, justamente impasse significa de dificil o imposible solucion, estos metodos al tratarse justamente de metodos predictores correctores fue necesario operarlos con cambio de variable y devolver el mismo para el grafico y obtencion de las soluciones correspondientes, tambien se logra ver como cierto desfase entre la formacion de esta asintota esto se puede deber a que como ya explicamos anteriormente en la resolucion por *Lsode* utilizamos una division uniforme y mas acaudalada en relacion a pasos del intervalo $[-1, 2]$ generandonos asi una grafica con mas definicion que la iteracion de *Heun* tomar en cuenta que el paso de *Heun* se realizo con la $Norma1$ o sea el valor absoluto de la diferencia de tiempos entre el paso $tiempos(i+1) - tiempos(i)$ por lo cual al tiempos tener 153 datos nos genera que estas diferencias sean un paso menos preciso que una division uniforme de una cantidad mas grande, debido a todo esto tambien es que se obtienen las diferencias de picos entre las distintas graficas ya que *Heun* por tener el paso mas grande puede saltearse el punto que hace la asintota mas visible. Tomar en cuenta que *Lsode* no es la misma solucion algoritmicamente hablando que *Heun* por eso debemos ser tolerantes a pequeñas diferencias.


Metodo de Heun para y0 = 1.0861 y Lsode para y0 = 1.0861
Heun
Lsode
Y value
Time
300
200
100
0
-100
-200
-300
-400
-1
-0.5
0
0.5
1
1.5
2


Metodo de Heun para y0 = 1.0832 y Lsode para y0 = 1.0832
Heun
Lsode
Y value
Time
300
200
100
0
-100
-200
-300
-400
-1
-0.5
0
0.5
1
1.5
2

Como se puede ver a continuacion las soluciones exactas para valores de tiempos nos dan exactamente igual a la solucion proporcionada por el comando *Lsode*.



Finalmente, hay que encontrar las 2 funciones que minimizen el error respecto a su correspondiente solucion de la segunda parte. Para esto, se calculo la diferencia en valor absoluta entre la funcion LSODE y Heun, promediando el error punto a punto. Las dos funciones que tengan el menor error promediado seran las que consideraremos. Aqui se observan las diferencias promedio de cada funcion:

```
>> Dif

dif1 = 3.2207e-03
dif2 = 0.013670
dif3 = 1.0788
dif4 = 0.9481
>> dif1 = dif2 = dif3 = dif4 = 0;
>> for i = 1:151
  dif1 = dif1 + abs(y1(i) - yLsode1(i+1));
  dif2 = dif2 + abs(y2(i) - yLsode2(i+1));
  dif3 = dif3 + abs(y3(i) - yLsode3(i+1));
  dif4 = dif4 + abs(y4(i) - yLsode4(i+1));
endfor
>> dif1 = dif1 /151
dif1 = 3.2207e-03
>> dif2 = dif2 /151
dif2 = 0.013670
>> dif3 = dif3 /151
dif3 = 1.0788
>> dif4 = dif4 /151
dif4 = 0.9481
>>
```

3.5. Interpolación con splines cúbicos

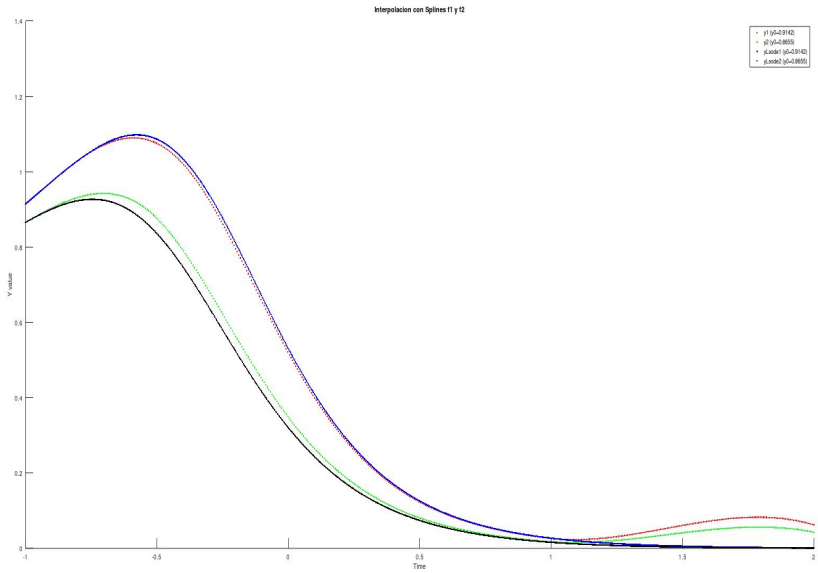
La implementación de este método empieza definiendo todas las variables utilizadas para el cálculo la variable iterativa $s_i(x_i)$, estos son:

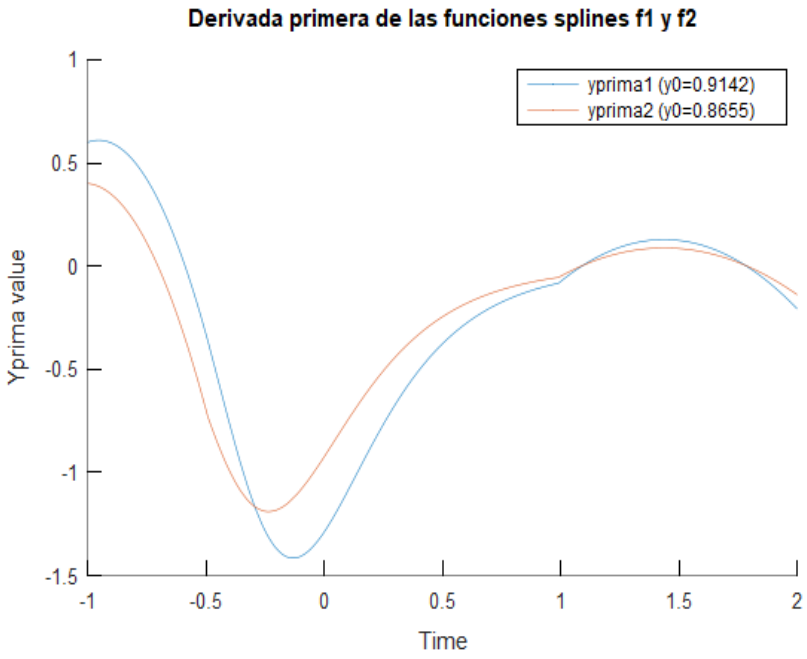
$$\begin{cases} h_i = x_{i+1} - x_i \\ \Delta_i = y_{i+1} - y_i \\ t = \frac{x - x_i}{h_i} \\ d_i = s_i'(x_i) \end{cases}$$

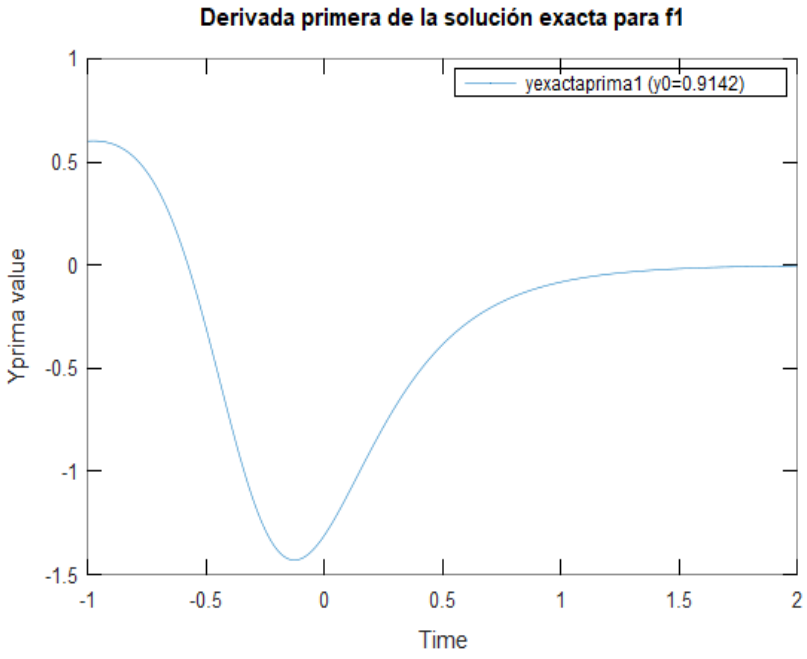
La idea principal del código es construir una función que implemente los splines (la decisión de hacerla una función radica en que, resulta más facil para la obtención de puntos en el armado de las gráficas). Dentro de la función, armamos un sistema lineal $Ad = b$, siendo b un vector con entradas de la forma $b = \frac{6\Delta_i}{h_i^2} + \frac{6\Delta_i}{h_i^2}$, d el vector de derivadas de los x_i y A una matriz tridiagonal con la digonal principal de la forma $a_{i,i} = \frac{4}{h_{i-2}} + \frac{4}{h_{i-1}}$ con $a_{1,1} = a_{n,n} = 1$, la diagonal superior $a_{i+1,i} = \frac{2}{h_{i-2}}$ con $a_{1,0} = 0$ y la diagonal inferior $a_{i-1,i} = \frac{2}{h_{i-1}}$ con $a_{n-1,n} = 0$. Una vez construido el sistema, lo resolvemos, obteniendo así el último dato que necesitabamos para poder calcular $s_i(x_i)$.

Una vez obtenidos todos los datos necesarios para calcular $s_i(x_i)$, mediante un for, calculamos todos los splines según la ecuación $s_i(x_i) = y_i(x_i) + t\Delta_i + t(t-1)(-h_i d_i) + t^2(t-1)(h_i(d_i + d_{i+1}) - 2\Delta_i$.

Las graficas de los resutados obtenidos son, la interpolación de splines con las condiciones menores a 1, sus gráficas respectivamente y la derivada de la función solución de la EDO. Esta última la ponemos para comparar la derivada obtenida en splines y la derivada real.




Derivada primera de las funciones splines f1 y f2
yprima1 (y0=0.9142)
yprima2 (y0=0.8655)
Yprima value
Time
1
0.5
0
-0.5
-1
-1.5
-1
-0.5
0
0.5
1
1.5
2


Derivada primera de la solución exacta para f1
yexactaprima1 (y0=0.9142)
Yprima value
Time
1
0.5
0
-0.5
-1
-1.5
-1
-0.5
0
0.5
1
1.5
2

3.6. Mínimos Cuadrados No Lineal

En esta parte del trabajo queremos ajustar a nuestros "datos experimentales", es decir, el conjunto $A = \{(t'_1, y(t'_1)), ..., (t'_n, y(t'_n))\}$ donde $t'_1, ..., t'_n$ son los proporcionados en $tiempos2.mat$ e $y(t)$ es la solución exacta del PVI con valor inicial $y0 = 0{,}9526$, a el modelo dado por $f(t) = \alpha g_1^{\beta}(t) + g_2^{\gamma}(t)$. Donde g_1 y g_2 son las fuciones $f_1(t)$ y $f_2(t)$ (funciones obtenidas de interpolar las soluciones de aplicar el método de $Heun$), las cuales minimizan el error respecto de las soluciones exactas. Las anteriores resultan de las soluciones con condiciones inciales $y0 = 0{,}9142$ e $y0 = 0{,}8655$ respectivamente. El modelo a ajustar puede apreciarse no lineal en los parámetros simplemente con observar la dependencia de los valores β y .

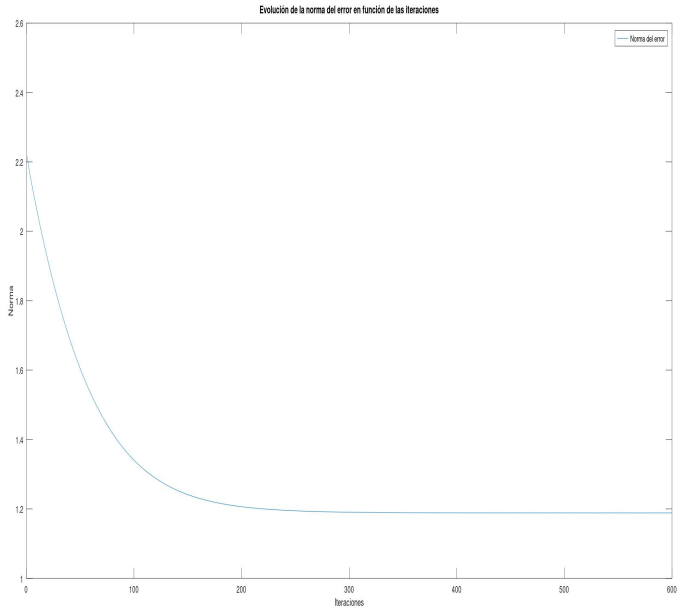
El algoritmo de Gauss-Newton modificado fue el implementado para el ajuste y se explica esta implementación mediante el siguiente seudocodigo:

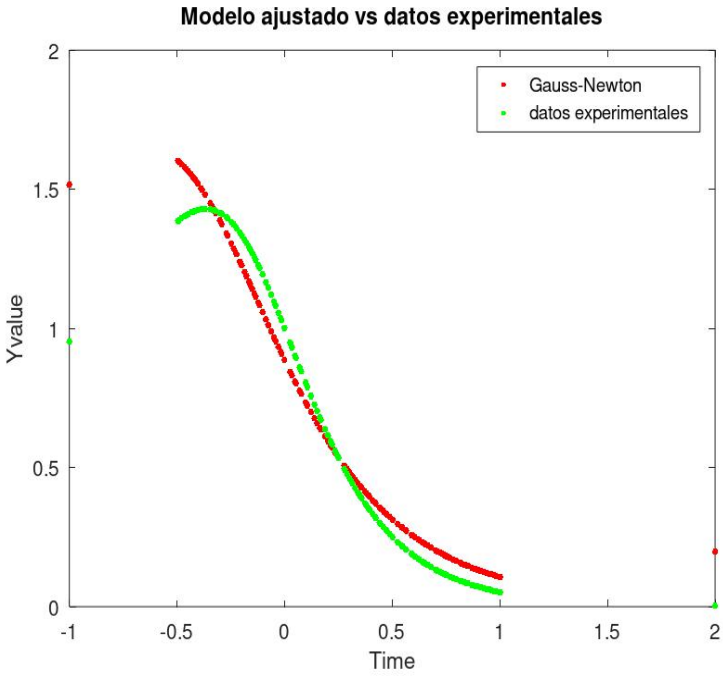
1-Definimos la función $F(X) = \alpha g_1^{\beta} + g_2^{\gamma}$ y su jacobiana $J_F(X) = [g_1^{\beta}, \alpha log(g_1)(g_1^{\beta}), log(g_2)(g_2^{\gamma})]$ con $X = (\alpha, \beta, \gamma)$ los parámetros utilizados.
2-Asignamos valores iniciales: $\alpha = \beta = \gamma = 1$ (es decir $X_k = (1, 1, 1)$), paso $\alpha_k = 0{,}01$ y límite de iteraciones $lim = 600$.

3-while i menor que 600:

$$
\begin{aligned}
A_k &= J_F(X_k) \\
Y_k &= y(t) - F(X_k) \\
&Resuelvoel PMCL \\
[Q, R] &= FactorizaciónQRdelamatrizA_k \\
P_k &= linsolve(R, Q^T Y_k) \\
&ActualizoX \\
X_k &= X_k + \alpha_k P_k \\
error(i) &= norm(Y_k)
\end{aligned}
$$

Siguiendo lo anterior obtenemos que el algoritmo converge en la iteración 476 a partir de la cual la norma del error se mantiene en 1.1887 con un tiempo de ejecución promedio aproximado de 1.14 segundos. Con valores finales de $\alpha = 0{,}65197$, $\beta = 0{,}78096$ y $\gamma = 0{,}66633$.





Como se puede ver en los anteriores gráficos se obtiene un muy buen ajuste dentro de los tiempos dados en el archivo tiempos2.mat.
Los siguientes son los valores hallados para la parte 6 del trabajo:

t	f(t)	y(t)
0.2	0.59678	0.61687
0.3	0.48348	0.46335
0.35	0.43444	0.39904
0.75	0.18238	0.11452

Tabla 1: Valores evaluados en $f(t)$ y comparación con la solución exacta (partes 6 y 7)

4. Conclusiones

Se logró resolver el PVI propuesto, mediante la implementación de métoodos numéricos de aproximación al resultado esperado.

Sin embargo, en un principio se observó que al implementar el Método de Heun para los 2 y_0 mayores a 1, el problema de la asíntota vertical en un cierto t provacaba que el método falle. Para solucionar esto se implementó el cambio de variable $u = 1/y$ y se lograron obtener resultados coherentes.

A su vez, comparando las gráficas de los métodos implementados con la curva teórica, hallada al resolver el PVI en la forma de Bernoulli mediante el método por Factor Integrante, se ve que tienen formas muy similares. También quedaron muy cercanas a las provistas al utilizar el comando de Octave para resolver este tipo de problemas, "Lsode".

Finalmente se pudo obtener buenas soluciones para otros valores experimentales (tiempos2) mediante la imlementación de del método de mínimos cuadrados, utilizando una función de una forma vinculada a las aproximaciones anteriores.

Referencias

[1] IMERL, *Apuntes de Teórico - Métodos Numéricos*. Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay, 2021

[2] The Octave Project Developers, *GNU Octave (version 7.2.0)*. 2022. https://docs.octave.org/interpreter/

[3] Lsode Docs, *Lsode documentation*. 2022. https://octave.sourceforge.io/octave/function/lsode.html

[4] *Spline interpolation*. Wikipedia. Recuperado el 10 de Noviembre, 2022 de https://en.wikipedia.org/wiki/Spline_interpolation

[5] *Heun's method*. Wikipedia. Recuperado el 10 de Setiembre, 2022 de https://en.wikipedia.org/wiki/Heun%27s_method#:~:text=Heun's%20Method%20considers%20the%20tangent,alone%2C%20approximated%20using%20Euler's%20Method.